



Sprint 5: Evaluation

Business Value, Dashboard y Documentación

Sprint Backlog, Roles y Definition of Done

Pedro Shiguihara, pshiguihara@pucp.edu.pe

8 de mayo de 2026

Agenda

1. Contexto: del modelo final al valor de negocio
2. Metodología: CRISP-DM y Scrum aplicados al Sprint 5
3. Product Backlog: ítems del Sprint 5
4. Sprint 5: Alcance y Sprint Planning
5. Roles del equipo (5 personas)
6. Detalle de tareas por rol
7. Entregables del Sprint 5
8. Criterios de aceptación y Definition of Done

Contexto del Sprint 5

Del Sprint 4 (modelo final) al Sprint 5 (evaluar valor)

🎯 Objetivo del Sprint 5

Evaluar el **valor de negocio** del modelo final entregado por el Sprint 4: traducir métricas técnicas a impacto económico, construir un **dashboard interactivo** para los stakeholders y **documentar** los hallazgos y recomendaciones que justifican el despliegue del MVP en el Sprint 6.

➡ Insumos del Sprint 4

- `12_hyperparam_tuning.ipynb` (modelos tuneados)
- `13_ensembles.ipynb` (ensambles y stacking)
- `14_final_validation.ipynb` (validación en test set)
- `src/tuning.py` (utilidades de tuning)
- `models/final_model.pkl` (modelo final versionado)
- `models/experiments_log.csv` (log completo)
- Métricas finales con intervalos de confianza

🛠 Herramientas Clave

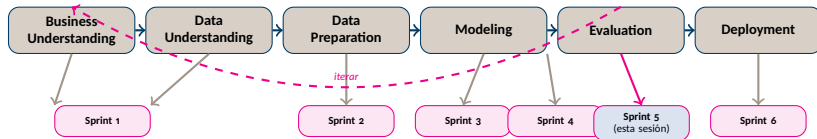
- 🐍 Python 3.10+
- 📊 Streamlit / Dash / Plotly
- 📊 NumPy + pandas (análisis costo-beneficio)
- 🔍 SHAP / LIME (interpretabilidad)
- 📄 Markdown + Quarto / Jupyter Book
- 💾 DVC (resultados versionados)
- 🔗 Git + GitHub Projects

⚠ Clave

Este sprint **no entrena modelos nuevos**: usa el modelo final del Sprint 4 y se enfoca en demostrar su **valor de negocio** y prepararlo para el despliegue en el Sprint 6.

Metodología: CRISP-DM dentro de Scrum

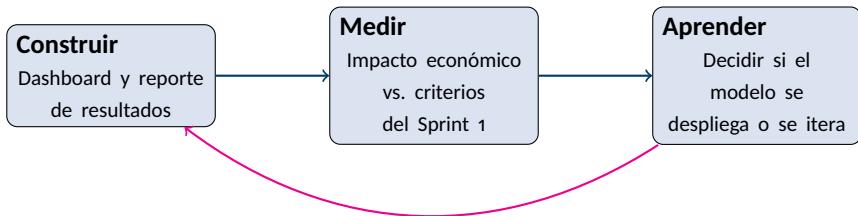
Mapeo de fases CRISP-DM a 6 Sprints — foco en Sprint 5



- **Sprint 1:** Business & Data Understanding.
- **Sprint 2:** Data Preparation.
- **Sprint 3:** Modeling baseline.
- **Sprint 4:** Modeling avanzado + tuning.
- **Sprint 5:** Evaluation + Business Value (*esta sesión*).
- **Sprint 6:** Deployment en AWS (MVP).

Ciclo Lean Startup aplicado al Sprint 5

Construir → Medir → Aprender



💡 Filosofía del Sprint 5

La fase de **Evaluation** de CRISP-DM no termina cuando el modelo es preciso: termina cuando el negocio entiende qué **decisiones tomar** con él. Este sprint convierte métricas en **recomendaciones** y comunica el valor del modelo a stakeholders no técnicos.

Product Backlog Completo

Historias de usuario para todo el proyecto (6 sprints)

ID	Historia de Usuario	Fase CRISP-DM	Sprint
PB-01	Definir el problema de negocio y objetivos	Business Underst.	1
PB-02	Identificar criterios de éxito del modelo	Business Underst.	1
PB-03	Cargar y explorar el dataset inicial	Data Understanding	1
PB-04	Analizar calidad de datos (nulos, tipos, duplicados)	Data Understanding	1
PB-05	Generar estadísticas descriptivas y visualizaciones	Data Understanding	1
PB-06	Construir prototipo interactivo de exploración	Data Understanding	1
PB-07	Limpiar y transformar variables	Data Preparation	2
PB-08	Realizar feature engineering	Data Preparation	2
PB-09	Balancear clases si es necesario	Data Preparation	2
PB-10	Entrenar modelos baseline de clasificación	Modeling	3
PB-11	Evaluar modelos baseline con métricas	Modeling	3
PB-12	Comparar modelos y seleccionar candidatos	Modeling	3
PB-13	Optimizar hiperparámetros del mejor modelo	Modeling	4
PB-14	Aplicar técnicas avanzadas (ensambles, stacking)	Modeling	4
PB-15	Validar modelo final (cross-validation, test set)	Modeling	4
PB-16	Evaluar Business Value del modelo seleccionado	Evaluation	5
PB-17	Construir dashboard interactivo de resultados	Evaluation	5
PB-18	Documentar hallazgos y recomendaciones de negocio	Evaluation	5
PB-19	Empaquetar modelo como API (Flask/FastAPI)	Deployment	6
PB-20	Desplegar prototipo MVP en AWS (EC2/Lambda)	Deployment	6
PB-21	Integrar modelo con dashboard desplegado	Deployment	6

→ Las filas resaltadas en azul corresponden al **Sprint 5** (esta iteración): PB-16, PB-17 y PB-18.

Sprint 5: Sprint Planning

Alcance: Evaluation — traducir métricas a valor de negocio

Meta del Sprint

Evaluar el **Business Value** del modelo final del Sprint 4, construir un **dashboard interactivo** para stakeholders y producir un **reporte ejecutivo** con hallazgos y recomendaciones que sustenten la decisión de desplegar el MVP.

Duración

- 1 semana de trabajo colaborativo
- Daily stand-ups (async vía GitHub Issues)
- Sprint Review con stakeholders al final de la semana

Sprint Backlog (3 ítems grandes)

- PB-16** Evaluar Business Value: traducir métricas técnicas a impacto económico (matriz costo-beneficio).
- PB-17** Construir dashboard interactivo (Streamlit/Dash) con KPIs, predicciones y explicabilidad.
- PB-18** Documentar hallazgos, riesgos y recomendaciones en un reporte ejecutivo.

Gestión

- Mismo repositorio GitHub de Sprints 1-4
- Cada sub-tarea = 1 Issue en el board Kanban
- Ramas por feature: `feat/PB-XX`

Roles del Equipo: 5 Personas

Roles adaptados al Sprint 5 (Evaluation)



Nota importante

Los roles **rotan** nuevamente respecto al Sprint 4: cada miembro del equipo asume un rol distinto para ganar experiencia en la fase de Evaluation. Todos colaboran y revisan el trabajo de los demás vía **Pull Requests**.

Rol 1: Project Manager

 Coordinación, planificación y seguimiento

Tareas del Sprint 5

1. Crear los Issues del Sprint 5 (PB-16, PB-17, PB-18) y sub-issues por tarea.
2. Reorganizar el board Kanban (cerrar Sprint 4, abrir Sprint 5).
3. Asignar responsables y revisores cruzados a cada Issue.
4. Verificar que el modelo final del Sprint 4 está disponible y reproducible.
5. Coordinar la secuencia: Business Value → dashboard → reporte.
6. Convocar al **stakeholder** (sponsor del proyecto) para el Sprint Review.
7. Preparar la demo del dashboard y el resumen del reporte ejecutivo.

Estructura del Repo (Sprint 5)

```
proyecto/  
+-- data/  
|   +-- raw/  
|   +-- processed/  
+-- notebooks/  
|   +-- 15_business_value.ipynb  
|   +-- 16_dashboard_prototype.ipynb  
|   +-- 17_findings_report.ipynb  
+-- src/  
|   +-- preprocessing.py  
|   +-- models.py  
|   +-- tuning.py  
|   +-- evaluation.py  
+-- dashboard/  
|   +-- app.py  
|   +-- pages/  
+-- reports/  
|   +-- final_report.pdf  
|   +-- recommendations.md  
+-- models/  
|   +-- final_model.pkl
```

Rol 2: Business Analyst

Evaluación de valor de negocio (PB-16)

Tareas del Sprint 5

1. Revisar los **criterios de éxito** definidos en el Sprint 1 (PB-02) y comparar contra las métricas del Sprint 4.
2. Construir la **matriz costo-beneficio**: costo de un Falso Positivo (FP) vs. Falso Negativo (FN) y beneficio de un True Positive (TP).
3. Calcular el **impacto económico anual estimado** del modelo (revenue gain / cost saving).
4. Calcular la **curva de ganancia** (gain curve) y el **lift** sobre la baseline aleatoria.
5. Identificar el **umbral óptimo** de decisión que maximiza el valor esperado de negocio.
6. Documentar supuestos, fuentes de los costos y rangos de sensibilidad.

Matriz Costo-Beneficio

```
import numpy as np
from sklearn.metrics import (
    confusion_matrix)

# Costos/beneficios definidos
# con el negocio
benefit_tp = 100 # USD ganados por TP
cost_fp = -20 # USD perdidos por FP
cost_fn = -80 # USD perdidos por FN
benefit_tn = 0 # neutral

y_pred = (y_proba >= 0.5).astype(int)
tn, fp, fn, tp = confusion_matrix(
    y_test, y_pred).ravel()

valor_total = (
    tp * benefit_tp + fp * cost_fp +
    fn * cost_fn + tn * benefit_tn)

print(f"Valor esperado: "
      f"USD {valor_total:,.0f}")
```

Rol 2: Business Analyst — Umbral óptimo

🔍 Maximizar valor esperado, no F1

? ¿Por qué ajustar el umbral?

- El umbral por defecto (0.5) maximiza **accuracy**, no el valor de negocio.
- Si el costo de un FN $\gg$ costo de un FP (p. ej. fraude), conviene un umbral **más bajo**.
- Si el costo de un FP $\gg$ costo de un FN (p. ej. campañas caras), conviene un umbral **más alto**.
- La elección del umbral es una **decisión de negocio**, no técnica: traducir la matriz costo-beneficio a un valor.

</> Búsqueda de umbral

```
import numpy as np
import pandas as pd

thresholds = np.arange(0.05, 0.95, 0.01)
results = []
for t in thresholds:
    y_pred = (y_proba >= t).astype(int)
    tn, fp, fn, tp = confusion_matrix(
        y_test, y_pred).ravel()
    valor = (
        tp*benefit_tp + fp*cost_fp +
        fn*cost_fn + tn*benefit_tn)
    results.append({'thr': t,
                   'valor': valor})

df = pd.DataFrame(results)
best = df.loc[df['valor'].idxmax()]
print(f"Umbral optimo: {best['thr']:.2f}")
print(f"Valor maximo: "
      f"USD {best['valor']:.0f}")
```

Rol 3: Dashboard Developer

 Dashboard interactivo de resultados (PB-17)

Tareas del Sprint 5

1. Elegir el framework: **Streamlit** (más simple) o **Dash** (más control).
2. Diseñar el wireframe: KPIs principales, tabla de predicciones y sección de explicabilidad.
3. Cargar el `models/final_model.pkl` y exponer un **simulador** de predicciones.
4. Visualizar la **confusion matrix** interactiva y la curva ROC.
5. Integrar las **explicaciones SHAP** por instancia para cada predicción.
6. Empaquetar el dashboard en `dashboard/app.py` listo para desplegar localmente.

Streamlit básico

```
import streamlit as st
import joblib
import pandas as pd

st.set_page_config(
    page_title="Modelo Final",
    layout="wide")

model = joblib.load(
    'models/final_model.pkl')

st.title("Dashboard del Modelo Final")
st.metric("F1 Score", "0.89", "+7.2%")
st.metric("AUC-ROC", "0.94", "+4.4%")

uploaded = st.file_uploader(
    "Sube un CSV", type='csv')
if uploaded:
    df = pd.read_csv(uploaded)
    preds = model.predict(df)
    df['pred'] = preds
    st.dataframe(df.head(20))
```

Rol 3: Dashboard Developer — Explicabilidad con SHAP

 ¿Por qué el modelo predice lo que predice?

¿Por qué SHAP?

- **Feature importance global** dice qué variables importan en promedio, pero no por qué una predicción específica fue 0 o 1.
- **SHAP** (SHapley Additive exPlanations) descompone cada predicción en contribuciones por feature.
- Permite responder al stakeholder: *“¿por qué este cliente fue clasificado como riesgo alto?”*
- Es clave para construir **confianza** con stakeholders no técnicos.
- Soporta visualizaciones: `summary_plot`, `force_plot`, `waterfall`.

SHAP en el dashboard

```
import shap
import streamlit as st
import streamlit.components.v1 as comps

explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(
    X_test)

# Resumen global
st.subheader("Importancia global")
fig = shap.summary_plot(
    shap_values, X_test, show=False)
st.pyplot(bbox_inches='tight')

# Explicacion individual
idx = st.slider("Instancia", 0,
                len(X_test)-1, 0)
force = shap.force_plot(
    explainer.expected_value,
    shap_values[idx], X_test.iloc[idx])
comps.html(shap.getjs() + force.html(),
           height=200)
```

Rol 4: Documentation Lead

Reporte ejecutivo y recomendaciones (PB-18)

Tareas del Sprint 5

1. Estructurar el **reporte ejecutivo** en secciones: contexto, datos, modelo, valor de negocio, riesgos y recomendaciones.
2. Redactar un **resumen ejecutivo** de 1 página para stakeholders sin background técnico.
3. Documentar los **riesgos** del modelo: sesgos, deriva (drift), supuestos críticos.
4. Incluir las **visualizaciones clave**: matriz de confusión, curva ROC, gain chart, SHAP summary.
5. Definir las **recomendaciones de negocio**: ¿desplegar el MVP? ¿con qué umbral? ¿para qué segmentos?
6. Generar el reporte final en PDF (Quarto / Jupyter Book) y publicarlo en `reports/final_report.pdf`.

Estructura del reporte

1. **Resumen ejecutivo** (1 página)
2. **Contexto del negocio** (Sprint 1)
3. **Datos y preparación** (Sprint 2)
4. **Modelo final** (Sprint 4)
 - Algoritmo y justificación
 - Hiperparámetros finales
 - Métricas + IC
5. **Valor de negocio** (Sprint 5)
 - Matriz costo-beneficio
 - Impacto económico estimado
 - Umbral óptimo
6. **Riesgos y limitaciones**
7. **Recomendaciones**
8. **Anexos técnicos**

Rol 4: Documentation Lead — Riesgos del modelo

 Comunicar los límites del modelo es tan importante como las métricas

Riesgos a documentar

- **Sesgos en los datos:** ¿hay subgrupos sub-representados?
- **Deriva (drift):** ¿qué pasará cuando los datos cambien?
- **Supuestos críticos:** ¿qué variables podrían no estar disponibles en producción?
- **Costos del modelo:** tiempo de inferencia, memoria, mantenimiento.
- **Cumplimiento:** ¿hay regulaciones (GDPR, sector financiero) aplicables?
- **Limitaciones de generalización:** el modelo se entrenó en un período y dataset específico.

Plantilla de recomendación

Para cada hallazgo o riesgo, redactar siguiendo este patrón:

- **Hallazgo:** ¿qué descubrió el equipo?
- **Evidencia:** ¿qué número o gráfico lo respalda?
- **Impacto:** ¿a qué área del negocio afecta?
- **Recomendación:** ¿qué acción concreta se sugiere?
- **Próximos pasos:** ¿quién, cómo y cuándo?

Ejemplo: “El modelo subestima el riesgo en clientes nuevos (recall=0.65 vs. 0.91 global). Recomendamos un umbral más bajo (0.35) para ese segmento durante los primeros 3 meses post-despliegue.”

Rol 5: QA & Stakeholder Reviewer



Validar calidad y preparar el handoff al Sprint 6

Tareas del Sprint 5

1. Revisar que los **números del reporte** coinciden con los notebooks (sin discrepancias entre fuentes).
2. Probar el **dashboard** de extremo a extremo: cargar CSV, simular predicciones, ver explicaciones SHAP.
3. Verificar que el **lenguaje del reporte** es accesible para stakeholders no técnicos.
4. Recopilar **feedback** de un stakeholder real (sponsor) en una reunión previa al Sprint Review.
5. Definir el **paquete de handoff** para el Sprint 6: modelo, preprocessing, requirements, ejemplos de input/output.
6. Coordinar con el PM la **agenda del Sprint Review** y ensayar la demo.

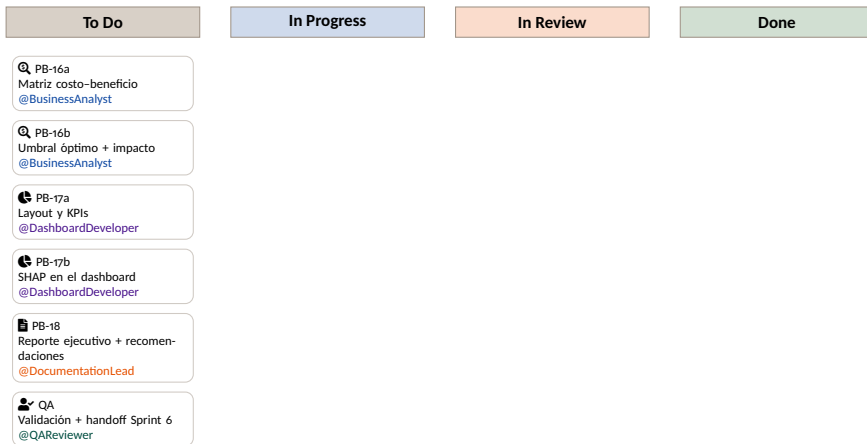
Paquete de handoff → Sprint 6

```
handoff/  
+-- model/  
|   +-- final_model.pkl  
|   +-- preproc_pipeline.pkl  
+-- contracts/  
|   +-- input_schema.json  
|   +-- output_schema.json  
|   +-- example_request.json  
|   +-- example_response.json  
+-- requirements.txt  
+-- README_handoff.md  
+-- KPIs_target_SLA.md
```

Este paquete es el **contrato** entre el Sprint 5 (qué entregamos) y el Sprint 6 (qué desplegamos). Sin esto, el deployment se vuelve adivinanza.

Tablero Kanban en GitHub Projects

Visualización del flujo de trabajo del Sprint 5



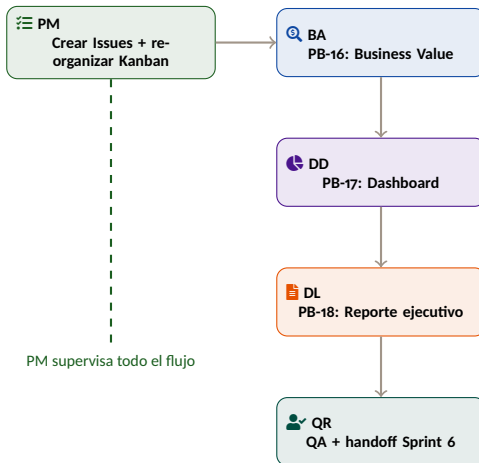
Flujo de Trabajo en el Kanban

Flujo

Cada Issue se mueve: **To Do** → se crea rama `feat/PB-XX` → **In Progress** → se abre Pull Request → **In Review** → se aprueba y mergea → **Done**. El QA & Stakeholder Reviewer integra al final, una vez el reporte y el dashboard están consolidados.

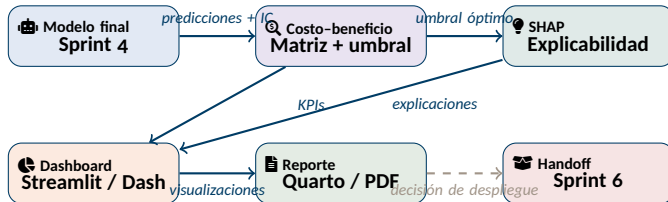
Dependencias entre Tareas

Orden sugerido de ejecución en el Sprint 5



Herramientas del Sprint 5: ¿Cómo encajan juntas?

Visión integrada para Evaluation



💡 Idea clave

El modelo final del Sprint 4 es técnicamente sólido. El Sprint 5 lo conecta con el negocio: traduce métricas a dólares, lo hace **interpretable** vía SHAP y construye los artefactos (dashboard + reporte) que sustentan la decisión de **desplegar** el MVP en el Sprint 6.

Tip 1: Métricas técnicas vs. métricas de negocio

⇌ Traducir F1 y AUC a dólares

¿Por qué no basta el F1?

- El F1 trata FP y FN como equivalentes; el negocio rara vez los trata así.
- Ejemplo fraude: un FN puede costar miles, un FP solo molesta a un cliente.
- Ejemplo marketing: un FP es un cupón desperdiciado; un FN es una venta perdida.
- El **valor esperado** (\$ por predicción) habla el idioma del stakeholder.
- La **curva de ganancia** muestra cuántos casos contactar para capturar X % del valor total.

💡 Regla práctica

Siempre presentar 2 versiones: (1) métricas técnicas para el equipo, (2) impacto en \$ y % de mejora vs. baseline para el negocio.

📈 Curva de ganancia

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Ordenar por probabilidad
order = np.argsort(-y_proba)
y_sorted = y_test[order]

# Acumulado de positivos
cum_pos = np.cumsum(y_sorted)
total_pos = y_test.sum()

x = np.arange(1, len(y_test)+1) / \
    len(y_test)
y_gain = cum_pos / total_pos

plt.plot(x, y_gain, label='Modelo')
plt.plot([0,1],[0,1], '--', label='Random')
plt.xlabel('% de poblacion contactada')
plt.ylabel('% de positivos capturados')
plt.legend()
plt.grid(True)
plt.show()
```

Tip 2: Streamlit vs. Dash — ¿cuál elegir?

 Comparativa para el dashboard

? Comparativa

Aspecto	Streamlit / Dash
Curva de aprendizaje	Streamlit (baja) / Dash (media)
Control fino del UI	Dash gana
Velocidad de prototipo	Streamlit gana
Multi-página	Streamlit (nativo) / Dash (callbacks)
Despliegue en AWS	Ambos (Docker + EC2/ECS)

💡 Recomendación

Para un **MVP rápido** en 1 semana, **Streamlit** es la opción más eficiente. Si el dashboard tendrá usuarios

21 concurrentes en producción a largo plazo, **Dash** ofrece más control.

</> Multi-página en Streamlit

```
# dashboard/app.py
import streamlit as st

st.set_page_config(
    page_title="Modelo Final",
    layout="wide")
st.title("Resumen")
st.write("KPIs principales aqui...")

# dashboard/pages/01_Predicciones.py
import streamlit as st
st.title("Predicciones")
# logica de prediccion

# dashboard/pages/02_Explicabilidad.py
import streamlit as st
st.title("SHAP por instancia")
# graficos SHAP

# Ejecutar:
# streamlit run dashboard/app.py
```

Tip 3: Storytelling con datos

🗣️ Cómo presentar el reporte ejecutivo

❓ Estructura “narrativa”

1. **Problema:** ¿qué dolor de negocio resolvemos?
2. **Datos:** ¿con qué contamos y qué nos falta?
3. **Modelo:** ¿qué construimos? (sin entrar en hiperparámetros)
4. **Resultados:** ¿qué tan bien funciona, en métricas y en \$?
5. **Riesgos:** ¿qué podría salir mal?
6. **Recomendación:** ¿qué proponemos hacer hoy?



Idea

Cada slide del reporte debe responder **una pregunta** del stakeholder. Si una slide no responde nada, sobra.

📄 Generar reporte con Quarto

```
# reports/final_report.qmd
---
title: "Modelo Final: Resultados"
author: "Equipo"
format:
  pdf:
    toc: true
    number-sections: true
---

## Resumen ejecutivo

El modelo logra un F1 de 0.89 y un
impacto economico estimado de
USD 1.2M anuales...

## Contexto

```{python}
import joblib
model = joblib.load(
 'models/final_model.pkl')
```

# Compilar:
# quarto render reports/final_report.qmd
```

Tip 4: Detección temprana de drift

 Anticipar cómo envejecerá el modelo en producción

? ¿Qué es el drift?

- **Data drift:** la distribución de las features cambia (nuevos clientes, estaciones del año, etc.).
- **Concept drift:** la relación entre features y target cambia (p. ej. nuevas regulaciones).
- Sin monitoreo, el modelo se degrada **silenciosamente**.
- El reporte del Sprint 5 debe **anticipar** qué variables son más susceptibles a drift y cómo el Sprint 6 las monitoreará.

</> Test KS para drift

```
from scipy.stats import ks_2samp

# Comparar train vs nueva ventana
def detectar_drift(X_ref, X_new,
                  alpha=0.05):

    drift = {}
    for col in X_ref.columns:
        stat, p = ks_2samp(
            X_ref[col], X_new[col])
        drift[col] = {
            'p_value': p,
            'drift': p < alpha,
        }
    return drift

# X_ref: dataset del Sprint 2
# X_new: ultimo lote en produccion
res = detectar_drift(X_train, X_recent)
print({k: v for k,v in res.items()
      if v['drift']})
```


Errores comunes en Evaluation

! Qué evitar y cómo resolverlo

| | Error común | Solución |
|---|--|---|
| ⊘ | Reportar solo F1/AUC sin traducir a \$ | Construir matriz costo-beneficio con el sponsor del proyecto |
| ⊘ | Mostrar dashboards sin explicabilidad | Integrar SHAP por instancia para construir confianza |
| ⊘ | Usar el umbral 0.5 sin justificarlo | Buscar el umbral que maximiza el valor esperado del negocio |
| ⊘ | Reporte técnico sin resumen ejecutivo | Una página al inicio que un CEO pueda leer en 90 segundos |
| ⊘ | No documentar riesgos y supuestos | Sección dedicada con sesgos, drift y limitaciones de generalización |
| ⊘ | Saltar al Sprint 6 sin handoff formal | Entregar paquete <code>handoff/</code> con modelo, contratos y SLAs |

Entregables del Sprint 5

Lo que cada rol debe entregar al final de la semana

| Rol | Entregable | Formato |
|-----|---|--------------------------------|
| PM | Issues del Sprint 5 cerrados, board Kanban actualizado, Sprint Review con stakeholder | GitHub Project |
| BA | Notebook con matriz costo-beneficio, umbral óptimo e impacto económico estimado | 15_business_value.ipynb |
| DD | Dashboard interactivo con KPIs, simulador de predicciones y SHAP por instancia | dashboard/app.py |
| DL | Reporte ejecutivo en PDF con resumen, hallazgos, riesgos y recomendaciones | reports/final_report.pdf |
| QR | Paquete handoff/ validado y feedback documentado del stakeholder | handoff/ + reports/feedback.md |

Definition of Done (DoD)

Criterios que cada tarea debe cumplir para considerarse terminada

✓ Criterios Generales

- ☐ Código en rama `feat/PB-XX` con Pull Request aprobado y mergeado a `main`.
- ☐ Notebook ejecutable de principio a fin sin errores (*Restart & Run All*).
- ☐ Celdas markdown documentando **cada decisión** de negocio.
- ☐ El entorno se reproduce con `conda env create -f environment.yml`.
- ☐ Issue correspondiente movido a "Done" en el board.

★ Criterios específicos del Sprint 5

- ☐ Matriz costo-beneficio firmada por el sponsor del proyecto.
- ☐ Umbral óptimo justificado con impacto económico cuantificado.
- ☐ Dashboard ejecutable localmente con `streamlit run` sin errores.
- ☐ Reporte ejecutivo con resumen de 1 página y recomendaciones accionables.
- ☐ Riesgos y supuestos críticos documentados explícitamente.
- ☐ Paquete `handoff` / listo para que el Sprint 6 inicie sin bloqueos.

Visión General: 6 Sprints hasta el Despliegue

Del entendimiento del negocio al MVP desplegado en AWS

| Sprint | Fase CRISP-DM | Objetivo principal |
|--------|--------------------------|--|
| 1 | Business + Data Underst. | Comprender el problema y explorar los datos |
| 2 | Data Preparation | Limpiar, transformar y crear features |
| 3 | Modeling | Entrenar modelos baseline y comparar |
| 4 | Modeling | Optimizar hiperparámetros y modelos avanzados |
| 5 | Evaluation | Evaluar Business Value y documentar resultados |
| 6 | Deployment | Desplegar el mejor modelo como MVP en AWS |

Meta del Sprint 5

Entregar la **evidencia de negocio** (impacto económico, dashboard y reporte) que sustenta la decisión de desplegar el MVP en AWS, más un **paquete de handoff** técnico que permita iniciar el Sprint 6 sin bloqueos.

Resumen del Sprint 5

¿Qué?

Evaluation: valor de negocio, dashboard y reporte

¿Quién?

5 roles: PM, BA, DD, DL, QR

¿Cómo?

Costo-beneficio + Streamlit + SHAP + Quarto



Entregable

Notebook business value + dashboard + reporte ejecutivo + paquete de handoff



Siguiente

Sprint 6: Deployment del MVP en AWS

! Recordar

- El Sprint 5 **no entrena modelos**: usa el modelo final del Sprint 4 y demuestra su valor.
- Cada decisión de negocio (umbral, segmento, KPI) se documenta y justifica con números.
- El stakeholder es la audiencia principal: el lenguaje y las visualizaciones deben ser para él.
- Sin paquete `handoff` /, el Sprint 6 se vuelve adivinanza: validar contratos y SLAs.

Checklist de Inicio

Pasos inmediatos para comenzar el Sprint 5

1. **PM:** Cerrar el Sprint 4 en el board y crear los Issues del Sprint 5 (PB-16, PB-17, PB-18 y sub-issues).
2. **PM:** Reasignar roles para que cada miembro experimente la fase de Evaluation.
3. **Todos:** `git pull && dvc pull` para sincronizar el estado del Sprint 4.
4. **Todos:** Verificar que el modelo final se carga correctamente:
`joblib.load('models/final_model.pkl')`.
5. **BA:** Iniciar `15_business_value.ipynb` en rama `feat/PB-16`, recoger costos con el sponsor y construir la matriz costo-beneficio.
6. **DD:** Inicializar `dashboard/app.py` en rama `feat/PB-17`, definir el wireframe y conectar el modelo.
7. **DL:** Crear `reports/final_report.qmd` (rama `feat/PB-18`) y armar la estructura del reporte ejecutivo.

Checklist de Inicio (cont.)

Pasos inmediatos para comenzar el Sprint 5

8. **QR:** Validar el dashboard y el reporte de extremo a extremo, recoger feedback del sponsor y armar el paquete `handoff` / (rama `feat/QA`).
9. **Todos:** Sprint Review con el sponsor — presentar dashboard, reporte, recomendación de despliegue y paquete de `handoff` para el Sprint 6.



Diplomado de Especialización en **Data Analytics**